CIA 2 Study Guide

End-to-End CI/CD Pipeline: Spring Boot, Maven, Docker, Jenkins, Ansible, Kubernetes on Google Cloud (GKE)

Pranav Surya R S · for personal study and viva preparation

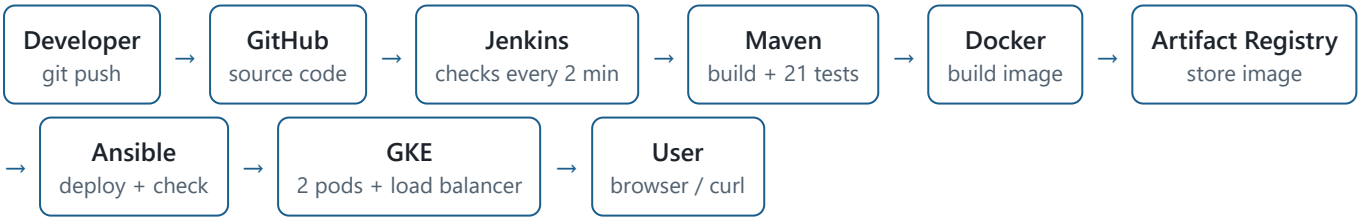
How to use this guide. Read section 1 first for the big picture. Then go file by file. Every config file is explained line by line in a table. Section 14 has likely viva questions with short answers. If you can answer those without looking, you are ready.

Contents

1. The big picture
2. Words you must know
3. The application
4. Maven: build and test
5. Docker: Dockerfile and .dockerignore
6. Artifact Registry (what an artifact is)
7. Kubernetes and GKE
8. deployment.yaml line by line
9. service.yaml line by line
10. Ansible: playbook and inventory
11. deploy.sh line by line
12. Jenkins and the Jenkinsfile
13. Google Cloud setup, problems, security, cost
14. Viva questions and answers
15. GitHub Actions (the actions branch)
16. Deeper questions explained
17. Jenkins webhook (instant builds)
18. Command cheat sheet

1. The big picture

The assignment asks for one thing: when a developer pushes code to Git, the app should be built, tested, packed into a container, stored in a registry and deployed to Kubernetes in the cloud, with nobody doing it by hand.



What happens on one push, in order:

1. You run `git push`. The new commit lands on GitHub (repo `pranavsurya77/devops-lab-cia2`, branch `main`).
2. Jenkins, running on your laptop, asks GitHub every 2 minutes "is there a new commit?". It sees one and starts a build. The build page says *Started by an SCM change*.
3. **Checkout SCM:** Jenkins copies the repo into its workspace folder.
4. **Build and Test:** `mvn clean package` compiles the Java code, runs all 21 tests and makes `target/library-manager.jar`. If one test fails, the pipeline stops here.

5. **Docker Build and Push:** Docker wraps the jar in an image, tags it with the build number (e.g. `:3`) and uploads it to Google Artifact Registry.
6. **Deploy with Ansible:** a Linux container logs in to the GKE cluster and runs the Ansible playbook. The playbook writes the new image tag into `deployment.yaml`, runs `kubectl apply`, waits for the new pods, finds the public IP and checks `/actuator/health` says UP.
7. Kubernetes replaces the old pods with new ones one at a time (rolling update). The load balancer IP stays the same, so users see the new version at the same address.

> **One-line answer for "explain your project":** A Spring Boot REST API whose every push to GitHub is picked up by a Jenkins pipeline that builds and tests it with Maven, packages it as a Docker image in Google Artifact Registry, and deploys it to a GKE Autopilot Kubernetes cluster using an Ansible playbook that also verifies the app is healthy.

Real numbers from our successful build (#3)

Stage	Time	Result
Checkout SCM	2 s	3 commits pulled from GitHub
Build and Test	16 s	Tests run: 21, Failures: 0, BUILD SUCCESS
Docker Build and Push	4 min 15 s	Image tag `3` in Artifact Registry (95.9 MB compressed)
Deploy with Ansible	3 min	PLAY RECAP ok=7 changed=4 failed=0, app at http://35.200.213.28
Total	**7 min 39 s**	**SUCCESS**

2. Words you must know

Term	Simple meaning
CI (Continuous Integration)	Every push is automatically built and tested, so broken code is caught at once.
CD (Continuous Deployment)	Code that passes is automatically released to the running environment.
Pipeline	The ordered list of automatic steps (stages) from code to running app.
Container	A small, isolated box that holds an app plus everything it needs (Java, libraries). Runs the same on any machine.
Image	The frozen template a container is started from. Built from a Dockerfile. Like a class; a container is like an object.
Tag	A label on an image version, e.g. `library-manager:3`. We use the Jenkins build number.
Registry	A server that stores images so other machines can download (pull) them. Ours: Google Artifact Registry.
Artifact	Any file a build produces: a jar, a Docker image, a package. Section 6.
Kubernetes (K8s)	A system that runs containers across machines, restarts them if they crash, scales them and gives them a network address.
Cluster	A group of machines (nodes) managed by Kubernetes. Ours is `cia2-gke`.

Node	One machine (VM) in the cluster that runs pods. In Autopilot, Google creates and manages nodes for us.
Pod	The smallest unit Kubernetes runs. Holds one (or more) containers. We run 2 pods, each with 1 container of our app.
Deployment	A Kubernetes object that says "keep N copies of this pod running" and handles rolling updates.
Service	A Kubernetes object that gives a fixed address to a changing group of pods and spreads traffic across them.
LoadBalancer	A Service type that asks the cloud for a public IP and a load balancer in front of the pods.
Manifest	A YAML file describing what you want in Kubernetes (our `deployment.yaml` and `service.yaml`).
kubectl	The command-line tool used to talk to a Kubernetes cluster.
GKE	Google Kubernetes Engine: Kubernetes run by Google.
Autopilot	A GKE mode where Google manages the nodes. You only describe pods.
Ansible	An automation tool. You write the steps once in a YAML **playbook** and Ansible runs them the same way every time.
Playbook	The Ansible YAML file with the list of tasks (ours: `ansible/deploy.yml`). Sometimes mis-said as "notebook".
Inventory	The Ansible file that lists which machines to run on (ours: `ansible/inventory.ini`).
Module	A built-in Ansible action, e.g. `replace`, `command`, `uri`, `debug`.
Jenkins	A CI/CD server that runs pipelines. Ours runs locally at http://localhost:8080.
Jenkinsfile	The pipeline written as code, stored in the repo.
SCM	Source Code Management, i.e. Git/GitHub. "Poll SCM" = check Git for changes.
Service account	A Google Cloud "robot user" for programs. Ours: `cia2-github`. Jenkins uses its key.
IAM role	A set of permissions given to a user or service account.
Rolling update	Replacing pods a few at a time so the app never fully goes down.

3. The application

Library Book Manager is a Spring Boot 3.3.5 REST API on Java 21. No UI, only JSON endpoints. Books are stored in memory (a `ConcurrentHashMap`), so there is no database to set up. It starts with 3 sample books.

Folder layout

```
cia2/
  pom.xml                      Maven build file (dependencies, Java version)
  src/main/java/com/cia2/library/
    LibraryApplication.java    main() that starts Spring Boot
    model/Book.java            the Book class (id, title, author, isbn, borrowedBy)
    dto/BookRequest.java       JSON body for adding a book (title, author required)
    dto/BorrowRequest.java     JSON body for borrowing (borrower required)
    service/BookService.java   all the rules (borrow, return, delete, search)
    controller/BookController.java   the /api/books endpoints
    controller/InfoController.java   / and /api/info (version + pod name)
    exception/...              404, 409 and 400 error handling
    config/OpenApiConfig.java  Swagger title and version
  src/main/resources/application.properties
  src/test/java/...            21 JUnit tests
  Dockerfile, .dockerignore
  k8s/deployment.yaml, k8s/service.yaml
  ansible/inventory.ini, ansible/deploy.yml, ansible/deploy.sh
  Jenkinsfile
```

Endpoints

Method	Path	What it does
GET	`/` or `/api/info`	App name, version (1.0.0) and host. In Kubernetes the host is the **pod name**, so you can see which pod answered.
GET	`/api/books?q=`	List books, or search by title or author
GET	`/api/books/{id}`	One book, 404 if missing
POST	`/api/books`	Add a book, 400 if title or author missing
POST	`/api/books/{id}/borrow`	Borrow, 409 if already borrowed
POST	`/api/books/{id}/return`	Return, 409 if not borrowed
DELETE	`/api/books/{id}`	Delete, 409 if borrowed out
GET	`/actuator/health`	Health check: `{"status":"UP"}`. Used by Ansible to confirm the deploy worked.
GET	`/swagger-ui.html`	Swagger page to try every endpoint in the browser

application.properties line by line

#	Line	Meaning
1	`spring.application.name=library-manager`	The app's name (shows in logs).

2	`server.port=8080`	The app listens on port 8080. This is why the Dockerfile says EXPOSE 8080 and the Service uses targetPort 8080.
5	`app.version=1.0.0`	Our own setting, shown at `/`. Change it to prove a redeploy worked.
8	`management.endpoints.web.exposure.include=health,info`	Spring Boot Actuator: only expose the `health` and `info` endpoints over HTTP (safer than exposing all).
9	`management.endpoint.health.probes.enabled=true`	Adds liveness and readiness groups (`/actuator/health/liveness`, `/readiness`) that Kubernetes probes can use.

The 21 tests

Test class	Count	What it checks
BookServiceTest	10	Business rules directly: seed books, add, search, borrow, can't borrow twice, return, can't return if not borrowed, delete, can't delete borrowed, unknown id gives "not found".
BookControllerTest	8	HTTP layer with MockMvc: correct status codes (200, 201, 204, 400, 404, 409) and JSON.
LibraryApplicationTests	3	Starts the whole app on a random port and calls `/api/info`, `/actuator/health` and `/v3/api-docs` for real.

4. Maven: build and test

Maven is the build tool for Java. `pom.xml` says which libraries we need (Spring Web, Validation, Actuator, springdoc for Swagger, Spring Boot Test) and that we use Java 21. Maven downloads them from Maven Central.

The command Jenkins runs: `mvn -B -ntp clean package`

Part	Meaning
`mvn`	Run Maven.
`-B`	Batch mode: no interactive prompts, no colours. Right for CI servers.
`-ntp`	"No transfer progress": hides the thousands of download progress lines so the Jenkins log is readable.
`clean`	Delete the old `target/` folder so nothing stale is reused.
`package`	Runs the phases in order: **compile** → **test** (runs all 21 tests) → **package** (builds the jar). If any test fails, Maven stops and the jar is not built.

The output is `target/library-manager.jar`, a "fat jar" that contains our code plus every library and an embedded Tomcat web server. That is why `java -jar app.jar` is all you need to run it. The `spring-boot-maven-plugin` in the pom makes this fat jar, and `<finalName>library-manager</finalName>` gives it a fixed name (no version number), which keeps the Dockerfile simple.

5. Docker: Dockerfile and .dockerignore

Dockerfile line by line

#	Line	Meaning and why
1	`FROM eclipse-temurin:21-jre-alpine`	Start from an official image that already has Java 21 (JRE = only what's needed to *run* Java, not compile). **Alpine** is a tiny Linux, so the image is small: 342 MB instead of 512 MB with the normal base. Smaller image = faster push/pull and it fits Artifact Registry's 0.5 GB free storage.
3	`COPY target/library-manager.jar app.jar`	Copy the jar Maven built into the image and call it `app.jar`. Maven runs **before** Docker in the pipeline, so the jar already exists.
5	`EXPOSE 8080`	Documents that the app listens on port 8080. It does not open anything by itself; Kubernetes uses `containerPort` and the Service for real networking.
7	`ENTRYPOINT ["java", "-jar", "app.jar"]`	The command run when the container starts: start the Spring Boot app.

> **Why so simple?** We first had a multi-stage Dockerfile that ran Maven inside Docker. Since Jenkins already builds the jar, building it twice is wasteful, so we simplified it to "copy the jar and run it". It is also much easier to explain.

.dockerignore

```
*
!target/library-manager.jar
```

When you run `docker build .`, Docker sends the folder (the "build context") to the Docker engine. `*` means "ignore everything", and `!target/library-manager.jar` means "except the jar". So only the jar is sent. This makes the build faster and makes sure source code, `.git`, secrets or test files never end up in the image.

The three Docker commands in the pipeline

Command	What it does
`docker build -t %IMAGE% .`	Build an image from the Dockerfile in the current folder (`.`) and tag it (`-t`) with the full Artifact Registry name, e.g. `asia-south1-docker.pkg.dev/cia2-513606/library-manager/library-manager:3`.
`type "%GCP_KEY%" | docker login -u _json_key --password-stdin https://asia-south1-docker.pkg.dev`	Log Docker in to Artifact Registry. `type` prints the JSON key file, the pipe `|` feeds it to `docker login` as the password (`--password-stdin`, so it never appears on the command line). `_json_key` is the special username Google uses for "the password is a service account JSON key".
`docker push %IMAGE%`	Upload the image layers to Artifact Registry.

6. Artifact Registry (what an artifact is)

An **artifact** is any file produced by a build that you keep and use later: a jar, a zip, an npm package, or a Docker image. **Google Artifact Registry** is Google Cloud's service for storing artifacts. We use it as a **Docker repository** to store our images. It replaced the older "Container Registry" (gcr.io).

Why we need it: the image is built on your laptop by Jenkins, but the pods run on Google's machines. The machines need somewhere to download the image from. That is the registry.

Why Artifact Registry and not Docker Hub: it is in the same Google Cloud project as the cluster, so GKE can pull images without any extra password setup, it stays private, and it is in the same region (Mumbai) so pulls are fast.

Anatomy of our image name

```
asia-south1-docker.pkg.dev / cia2-513606 / library-manager / library-manager : 3
        |                        |               |                 |          |
  registry host (region)     project ID     repository name     image name   tag (Jenkins build
number)
```

- **Repository** `library-manager`: created once in the console, format *Docker*, region *asia-south1*.
- **Tag** `3`: each Jenkins build pushes a new tag (1, 2, 3...), so every version is kept and can be traced to a build.
- **Digest** (e.g. `sha256:568ea9...`): the unique fingerprint of the image content. Tags can move; digests never change.
- Size in the console shows **95.9 MB** because images are stored compressed. Locally Docker shows 342 MB uncompressed.

7. Kubernetes and GKE

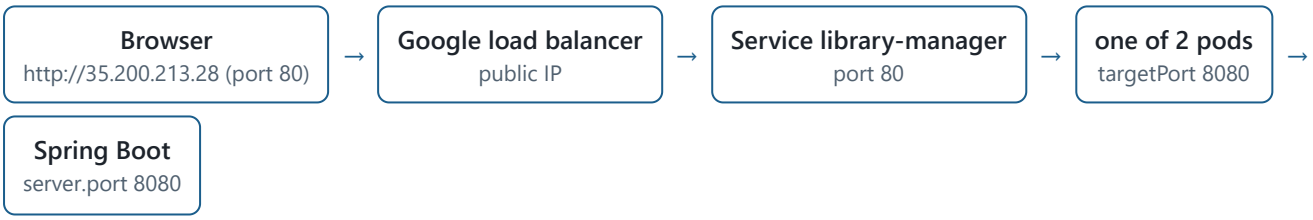
Why Kubernetes? Running one container is easy. Running it reliably is not. Kubernetes keeps the number of copies you asked for, restarts crashed containers, replaces pods one at a time during an update, and gives the pods a stable network address. You describe the *desired state* in YAML; Kubernetes keeps making reality match it.

GKE Autopilot: Google runs the Kubernetes control plane *and* the worker nodes. We never chose a machine size; Autopilot adds nodes when pods need them (that is why the first deploy took a few minutes). The cluster page showed **1 node, 1 vCPU, 4 GB** total. Our cluster: name `cia2-gke`, region `asia-south1` (Mumbai), project `cia2-513606`.

Why Autopilot: fewest setup steps, and the GKE free tier gives a $74.40 monthly credit that covers the management fee ($0.10/hour) of one Autopilot or zonal cluster. You still pay for pod resources and the load balancer, which our credit covered.

> **How many pods does ours create? 2.** Because `replicas: 2` in `deployment.yaml`. Each pod runs 1 container of our app. The Workloads page shows **2/2**. During an update there are briefly **3** pods: Kubernetes' default rolling update allows 25% extra (rounded up = 1 extra pod) and 25% unavailable (rounded down = 0), so it starts 1 new pod, waits for it, then removes 1 old pod, and repeats. The app never goes fully down.

How a request reaches the app



The Service picks pods by **label** (`app: library-manager`), not by name. Pod names change every deploy (e.g. `library-manager-5f6c76744b-rbf6c`), but the label stays the same, so the Service always finds the current pods.

8. deployment.yaml line by line

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: library-manager
spec:
  replicas: 2
  selector:
    matchLabels:
      app: library-manager
  template:
    metadata:
      labels:
        app: library-manager
    spec:
      containers:
        - name: library-manager
          image: library-manager:local
          ports:
            - containerPort: 8080
```

#	Line	Meaning
1	`apiVersion: apps/v1`	Which Kubernetes API group/version this object uses. Deployments live in `apps/v1`.
2	`kind: Deployment`	The type of object: a Deployment (keeps N pods running, does rolling updates).
3	`metadata:`	Information about the object itself.
4	`  name: library-manager`	Its name. Used in commands like `kubectl rollout status deployment/library-manager`.
5	`spec:`	The desired state: what we want.
6	`  replicas: 2`	**Keep 2 pods running at all times.** If one crashes or its node dies, Kubernetes starts a replacement. 2 gives basic high availability and lets the rolling update work without downtime.
7	`  selector:`	How the Deployment knows which pods belong to it...
8-9	`    matchLabels:` `      app: library-manager`	...any pod with the label `app=library-manager`. This must match the labels in the template (line 13), otherwise Kubernetes rejects the file.
10	`  template:`	The blueprint used to create each pod.

11-13	`    metadata:`
	`      labels:`
	`        app: library-manager`
		Every pod gets the label `app=library-manager`. The Deployment (line 9) and the Service (service.yaml) both find pods using this label.

14	`    spec:`	What runs inside each pod.
15	`      containers:`	List of containers in the pod. We have one.

16	`- name: library-manager`	The container's name inside the pod.
17	`image: library-manager:local`	Which image to run. **This is a placeholder.** The Ansible playbook replaces it with the real Artifact Registry image and tag (e.g. `...library-manager:3`) on every deploy. A new image value is what makes Kubernetes roll out new pods. `:local` was used when we tested on a local kind cluster.
18-19	`ports:` `  - containerPort: 8080`	The container listens on 8080 (matches `server.port`). Mostly informational; the Service's `targetPort` does the real routing.

> **Kept simple on purpose.** We removed resource limits and health probes from this file so it is easy to explain. Downsides you can mention in the viva: without a readiness probe, Kubernetes sends traffic as soon as the container starts (Spring Boot needs a few seconds); without resource requests, Autopilot applies its own defaults. In production you would add `readinessProbe` on `/actuator/health/readiness` and `resources`.

9. service.yaml line by line

```
apiVersion: v1
kind: Service
metadata:
  name: library-manager
spec:
  type: LoadBalancer
  selector:
    app: library-manager
  ports:
    - port: 80
      targetPort: 8080
```

#	Line	Meaning
1	`apiVersion: v1`	Services are in the core API group, version v1.
2	`kind: Service`	A Service: a stable address in front of a set of pods.
4	`  name: library-manager`	The Service's name. Ansible reads its IP with `kubectl get service library-manager`.
6	`  type: LoadBalancer`	Ask the cloud for a **public IP and load balancer**. On GKE this creates a Google Cloud load balancer; we got `35.200.213.28`. Other types: `ClusterIP` (default, only inside the cluster) and `NodePort` (a port on every node).
7-8	`  selector:` `    app: library-manager`	Send traffic to every pod with this label, i.e. our 2 pods. Traffic is spread between them.
10	`    - port: 80`	The port users connect to on the Service/load balancer. 80 is the default HTTP port, so no `:8080` in the URL.
11	`      targetPort: 8080`	The port on the pod to forward to, where Spring Boot listens.

> **Why the IP survives redeploys:** Ansible re-applies the Service each time, but it is unchanged, so Kubernetes keeps the same load balancer and IP. Only the pods behind it change. Deleting the Service (as we did to save money) removes the load balancer, and the next deploy may get a new IP.

10. Ansible: playbook and inventory

What it is: Ansible is an automation tool for configuration and deployment. You write *what to do* as a list of tasks in a YAML **playbook**, and Ansible runs them in order, the same way every time. It is **agentless**: nothing needs to be installed on the target machines; it connects over SSH (or runs locally, as in our case).

Why we need it: the assignment says to apply basic Ansible for deployment automation. Deploying is several steps (set the image, apply manifests, wait for pods, find the IP, test health). Putting them in a playbook makes them repeatable, readable and reusable outside Jenkins too. Without it, those steps would be loose shell commands in the Jenkinsfile.

How we set it up: Ansible does not run on Windows, and Jenkins runs on Windows. So nothing is installed on your laptop. Each deploy starts a Linux container (`google/cloud-sdk:slim`), installs `ansible-core` inside it with `apt-get` , and runs the playbook there (section 11). The container is deleted afterwards (`--rm`).

inventory.ini line by line

#	Line	Meaning
1	`[local]`	A group of hosts named `local` . The playbook targets this group.
2	`localhost ansible_connection=local`	The only host is this machine (the container itself). `ansible_connection=local` means "run tasks directly here, do not SSH". We only need `kubectl` to talk to GKE, and kubectl runs locally.

deploy.yml line by line

#	Line	Meaning
1	`- name: Deploy Library Manager to Kubernetes`	A **play** (a block of tasks for some hosts). The name is printed in the log as `PLAY [...]` .
2	`  hosts: local`	Run on the `local` group from the inventory.
3	`  vars:`	Variables for this play.
4	`    image: library-manager:local`	Default image. Jenkins overrides it with `-e image=...:3` (extra vars always win).
5	`    k8s_dir: "{{ playbook_dir }}/../k8s"`	Path to the k8s folder. `{{ }}` is Jinja2 templating; `playbook_dir` is the folder the playbook is in, so this works from anywhere.
7	`  tasks:`	The steps, run top to bottom. If one fails, the play stops (and so does Jenkins).

8-12	`- name: Put the new image into deployment.yaml`
	`  ansible.builtin.replace:`
	`    path: ".../deployment.yaml"`
	`    regexp: "image: .*"`
	`    replace: "image: {{ image }}"`
		The `replace` module finds the line matching the regex `image: .*` ("image:" followed by anything) and rewrites it with the new image. This is how each deploy points Kubernetes at the new version.

14-15	`- name: Apply the Kubernetes manifests` `  ansible.builtin.command: kubectl apply -f {{ k8s_dir }}`	Runs `kubectl apply` on the whole folder (both YAML files). `apply` creates objects if missing, or updates them to match the files. The changed image triggers a rolling update.
17-18	`- name: Wait until the new pods are running` `  command: kubectl rollout status deployment/library-manager --timeout=600s`	Blocks until the rolling update finishes (all new pods ready), or fails after 10 minutes. 600 s because Autopilot may first need to create a node.

20-25	`- name: Get the app's public IP`
	`  command: kubectl get service library-manager -o jsonpath={.status.loadBalancer.ingress[0].ip}`
	`  register: ip`
	`  until: ip.stdout != ""`
	`  retries: 30`
	`  delay: 10`
		Reads the load balancer's external IP from the Service. `jsonpath` picks one field out of the JSON. `register: ip` saves the output in a variable. A new load balancer takes a minute to get an IP, so `until/retries/delay` retries up to 30 times, 10 s apart (5 min max) until the output is not empty.

27-33	`- name: Check the app is healthy`
	`  ansible.builtin.uri:`
	`  url: "http://{{ ip.stdout }}/actuator/health"`
	`  register: health`
	`  until: health.status == 200`
	`  retries: 30`
	`  delay: 10`
		The `uri` module makes an HTTP request (like curl). It retries until `/actuator/health` returns HTTP 200. This is the "verify deployment and application availability" stage. In build #3 it failed 3 times while pods started, then succeeded.

35-37	`- name: Show the app URL`
	`  ansible.builtin.debug:`
	`  msg: "App is running at http://{{ ip.stdout }}"`
		Prints the URL in the log, e.g. `App is running at http://35.200.213.28`.

Reading the PLAY RECAP: `ok=7 changed=4 failed=0`

- **ok=7**: 7 tasks succeeded = 6 of ours + "Gathering Facts" (Ansible collects info about the host first).
- **changed=4**: replace, apply, rollout status and get IP. The `command` module always reports "changed" because Ansible can't know what a command did. `uri` and `debug` only read, so they report "ok".
- **failed=0**: nothing failed. **unreachable=0**: the host was reachable.

11. deploy.sh line by line

This small script runs **inside** the `google/cloud-sdk:slim` container. It prepares the tools, logs in to the cluster, then runs Ansible.

#	Line	Meaning
1	`#!/bin/bash`	Run this file with bash.
3	`set -e`	Stop at the first command that fails. The container then exits with an error and Jenkins marks the stage as failed.
6	`cp -r /work /tmp/app && cd /tmp/app`	Copy the repo (mounted from the Jenkins workspace at `/work`) into the container and work on the copy. Without this, Ansible's edit of `deployment.yaml` failed with "Permission denied" on the Windows-mounted folder (build #1).
8	`apt-get update -qq && apt-get install -y -qq ansible-core kubectl google-cloud-cli-gke-gcloud-auth-plugin > /dev/null`	Install 3 tools quietly: **ansible-core** (runs the playbook), **kubectl** (talks to Kubernetes), **gke-gcloud-auth-plugin** (lets kubectl log in to GKE using Google credentials; required for GKE).
10	`gcloud auth activate-service-account --key-file=/key.json`	Log the `gcloud` tool in as the `cia2-github` service account, using the JSON key Jenkins mounted at `/key.json`.
11	`gcloud container clusters get-credentials cia2-gke --region asia-south1 --project cia2-513606`	Fetch the cluster's address and certificate and write a kubeconfig (`~/.kube/config`), so `kubectl` now points at our GKE cluster.
13	`ansible-playbook -i ansible/inventory.ini ansible/deploy.yml -e image="$1"`	Run the playbook with our inventory (`-i`). `-e image="$1"` passes the image name, which Jenkins gave as the script's first argument (`$1`).

12. Jenkins and the Jenkinsfile

What Jenkins is: an open-source automation server for CI/CD. It watches the repo and runs the pipeline. Ours is version 2.568.2, installed on your laptop as a **Windows service** running as the **LocalSystem** account, at `http://localhost:8080`.

How we set it up:

1. **Credential:** Manage Jenkins → Credentials → added the service account JSON key as a **Secret file** with ID `gcp-key`.
2. **Job:** a **Pipeline** job named `cia2-pipeline` with "Pipeline script from SCM": Git, URL `https://github.com/pranavsurya77/devops-lab-cia2.git`, branch `*/main`, script path `Jenkinsfile`. The repo is public, so no GitHub login was needed.
3. **First run:** clicked Build Now once. That first run registers the `pollSCM` trigger; after that, pushes start builds by themselves.

Why Jenkins (and not GitHub Actions): we first used GitHub Actions, then switched to Jenkins because the pipeline should be simple, local and fully under our control. The Jenkinsfile lives in the repo, so the pipeline is

versioned with the code ("pipeline as code").

Jenkinsfile line by line

#	Line	Meaning and why
1	`pipeline {`	Start of a **declarative** pipeline (the simpler, structured Jenkinsfile style).
2	`    agent any`	Run on any available Jenkins agent. We only have the built-in one (the laptop).

4-7	`    triggers {`
	`        pollSCM('H/2 * * * *')`
	`    }`
		Every 2 minutes, ask GitHub if `main` has a new commit; if yes, start a build. Cron format: minute hour day month weekday. `H/2` = every 2 minutes, with `H` (hash) spreading jobs so they don't all poll at the same second. We poll instead of a GitHub webhook because a laptop has no public URL GitHub could call.

9	`    environment {`	Environment variables for all stages.
10	`        PATH = "C:\\tools\\apache-maven-3.9.16\\bin;...DockerDesktop\\resources\\bin;${env.PATH}"`	Adds the Maven and Docker folders to `PATH`. The Jenkins service runs as LocalSystem, which does not have your user PATH, so without this `mvn` and `docker` are "not recognized". `\\` is an escaped backslash in Groovy strings.
11	`        IMAGE = "asia-south1-docker.pkg.dev/cia2-513606/library-manager/library-manager:${BUILD_NUMBER}"`	The full image name used by every stage. `BUILD_NUMBER` is set by Jenkins (1, 2, 3...), giving each build a unique tag.
12	`        GCP_KEY = credentials('gcp-key')`	Loads the secret file credential. Jenkins writes the key to a temporary file and puts its **path** in `GCP_KEY`. Jenkins masks it as `****` in logs and deletes it after the build.
15	`    stages {`	The ordered stages. Jenkins also adds a **Checkout SCM** stage at the start automatically (clones the repo) because the job is "from SCM".

16-20	`stage('Build and Test') {`
	`  steps { bat 'mvn -B -ntp clean package' }`
	`}`
		Compile, run 21 tests, build the jar (section 4). `bat` runs a Windows batch command (on Linux agents you'd use `sh`). A failing test returns a non-zero exit code, which fails the build.

22-28	`stage('Docker Build and Push') {`
	`  bat 'docker build -t %IMAGE% .'`
	`  bat 'type "%GCP_KEY%" | docker login -u _json_key --password-stdin https://asia-south1-docker.pkg.dev'`
	`  bat 'docker push %IMAGE%'`
	`}`
		Build the image, log in to Artifact Registry with the key, push (section 5). `%IMAGE%` is Windows batch syntax for reading an environment variable.

30-34	`stage('Deploy with Ansible') {`
	`  bat 'docker run --rm -v "%WORKSPACE%:/work" -v "%GCP_KEY%:/key.json" -w /work google/cloud-sdk:slim bash ansible/deploy.sh %IMAGE%'`
	`}`
		Start a Linux container and run `deploy.sh` in it. `--rm`: delete the container when done. `-v "%WORKSPACE%:/work"`: mount the Jenkins workspace (the repo) at `/work`. `-v "%GCP_KEY%:/key.json"`: mount the key file. `-w /work`: start in that folder. `google/cloud-sdk:slim`: Google's official image with `gcloud`. `%IMAGE%` becomes `$1` in the script.

Why each stage exists

Stage	Why it's needed
Checkout SCM	Jenkins needs the latest code before it can build it.
Build and Test	Catch broken code early. Nothing is deployed unless all 21 tests pass.
Docker Build and Push	Package the app with its Java runtime into one portable image and put it where the cloud can download it.
Deploy with Ansible	Update the running app on Kubernetes, then prove it works (health check), so a "green" build means the app is really live.

13. Google Cloud setup, problems, security, cost

What we set up in the Google Cloud console, and why

Step	What	Why
1	Project `cia2` (ID `cia2-513606`) linked to the billing account	Everything in Google Cloud lives in a project; billing is needed even for free-tier use.
2	Budget alert	Email warning before any real cost.
3	Enabled Kubernetes Engine API and Artifact Registry API	APIs are off by default in a new project.
4	Artifact Registry repo `library-manager` (Docker, asia-south1)	Somewhere to push images.
5	GKE Autopilot cluster `cia2-gke` (asia-south1)	Where the pods run. Autopilot = least setup, free-tier management fee.
6	Service account `cia2-github` + JSON key	A robot identity for the pipeline, so we never use a personal Google login in CI.
7	Roles: **Artifact Registry Writer** and **Kubernetes Engine Developer**	Least privilege: push images, and deploy to the cluster. Nothing else. (Our first push failed with `uploadArtifacts denied` until these roles were added.)

The name `cia2-github` is left over from when GitHub Actions used it; Jenkins now uses the same account.

Problems we hit and how we fixed them

Problem	Cause	Fix
App would not start locally	Port 8080 used by Jenkins	Ran locally with `--server.port=8081`
Image push "permission denied"	Service account had no IAM roles	Granted Artifact Registry Writer + Kubernetes Engine Developer
"mvn / docker is not recognized"	Jenkins service (LocalSystem) has no user PATH	Set PATH in the Jenkinsfile `environment`
Can't run Ansible	Ansible does not support Windows as a control node	Run it inside the Linux `google/cloud-sdk` container
Ansible "Permission denied" editing deployment.yaml (build #1 failed)	Can't replace files in a Windows folder mounted into Linux	`deploy.sh` copies files to `/tmp/app` first (build #2 passed)
Image 512 MB, over free storage	Full JRE base image	Alpine JRE base: 342 MB (96 MB compressed)
Cost and account issues on AWS/Azure	EKS charges $0.10/h with unclear free-plan support; Azure setup failed	GKE Autopilot with the GKE free tier

Security

- The GCP key is never in Git. It is a Jenkins **secret file** credential, shown as `****` in logs, passed to Docker through stdin.
- The service account has only 2 roles (least privilege).
- `.dockerignore` keeps everything except the jar out of the image; the runtime image is a minimal Alpine JRE.
- Only the `health` and `info` actuator endpoints are exposed.
- Each image tag maps to one Jenkins build, so any running version can be traced.

Limitations (good to admit in the viva)

- In-memory data: books reset when a pod restarts, and the 2 pods don't share data. Fix: a database.
- Plain HTTP and no login on the API. Fix: Ingress with TLS, API authentication.
- Polling every 2 minutes instead of a webhook (laptop has no public URL).
- A long-lived JSON key is used. Better: Workload Identity Federation.
- No readiness probe or resource limits in deployment.yaml (kept simple).
- Jenkins depends on your laptop and Docker Desktop being on.

Cost and how to stop it

- GKE cluster management fee: covered by the free tier ($74.40/month credit, one Autopilot cluster).
- Billed while running: the pods (Autopilot charges per pod resources) and the load balancer.
- **To stop charges:** Workloads → library-manager → Scale to 0, and delete the Service (removes the load balancer). To remove everything: delete the cluster `cia2-gke`. A new Jenkins build re-creates the Service and 2 pods.

14. Viva questions and answers

Q1. What does your pipeline do?
On every push to GitHub, Jenkins checks out the code, builds and tests it with Maven, builds a Docker image, pushes it to Artifact Registry, and an Ansible playbook deploys it to GKE and checks it is healthy.

Q2. How is the pipeline triggered?
By `pollSCM('H/2 * * * *')`: Jenkins checks GitHub every 2 minutes and builds when there is a new commit. The build says "Started by an SCM change".

Q3. Why polling and not a webhook?
A webhook needs GitHub to reach Jenkins over the internet. Our Jenkins is on a laptop with no public URL, so Jenkins asks GitHub instead.

Q4. What happens if a test fails?
Maven exits with an error, the Build and Test stage fails, and the pipeline stops. No image is built and nothing is deployed.

Q5. How many pods run, and why?
2, set by `replicas: 2`. Two copies give availability if one dies, and allow a rolling update with no downtime. Briefly 3 during an update.

Q6. Difference between a Deployment and a Service?
A Deployment runs and manages the pods (how many, which image, updates). A Service gives those pods one stable address and load-balances traffic to them.

Q7. How does the Service know which pods to send traffic to?
By label selector `app: library-manager`, which matches the label in the Deployment's pod template.

Q8. Explain port 80 and 8080.
Users hit port 80 on the load balancer (`port`). The Service forwards to port 8080 on the pod (`targetPort`), where Spring Boot listens (`server.port=8080`).

Q9. Why type LoadBalancer?
It gives a public IP so the app is reachable from a browser. ClusterIP would only be reachable inside the cluster.

Q10. What is an artifact? What is Artifact Registry?
An artifact is a build output, like a jar or Docker image. Artifact Registry is Google Cloud's private storage for them; we keep our Docker images there so GKE can pull them.

Q11. How does a new version get deployed?
Jenkins tags the image with the build number. Ansible writes that tag into deployment.yaml and runs `kubectl apply`. The image changed, so Kubernetes does a rolling update.

Q12. What is a rolling update?
Pods are replaced a few at a time: start 1 new, wait until ready, stop 1 old, repeat. Users never see downtime.

Q13. What is Ansible and why did you use it?
An agentless automation tool that runs YAML playbooks. We used it to make deployment repeatable: set image, apply manifests, wait for rollout, get IP, health check.

Q14. What is an inventory? What is in yours?
The list of hosts Ansible manages. Ours has one host, `localhost` with `ansible_connection=local`, because the tasks only run kubectl from the same container.

Q15. Why does Ansible run inside a Docker container?
Jenkins runs on Windows, and Ansible can't run on Windows as a control node. The `google/cloud-sdk` Linux container also gives us gcloud and kubectl.

Q16. What do until, retries and delay do?
They repeat a task until the condition is true: up to 30 tries, 10 seconds apart. Used to wait for the public IP and for health to return 200.

Q17. How does Jenkins get permission to push and deploy?
It uses the `cia2-github` service account's JSON key, stored as the Jenkins secret file credential `gcp-key`. The account has only Artifact Registry Writer and Kubernetes Engine Developer roles.

Q18. How do you keep the key secret?
It is never in Git. `credentials('gcp-key')` gives a temp file path, Jenkins masks it in logs, and Docker receives it via `--password-stdin`.

Q19. Why the Alpine base image?
It is much smaller (342 MB vs 512 MB), so push and pull are faster and it fits the free registry storage.

Q20. Why does the Dockerfile only copy a jar?
Jenkins already built and tested the jar with Maven, so the image just needs Java and the jar. Simple and fast.

Q21. What is GKE Autopilot?
A GKE mode where Google manages the nodes. We only define pods; Google adds machines as needed and bills per pod.

Q22. How do you verify the deployment?
`kubectl rollout status` waits for the new pods, then Ansible calls `/actuator/health` and requires HTTP 200 with status UP. We also checked with `curl` and the GKE console (2/2 pods, OK).

Q23. What is Spring Boot Actuator?
A Spring Boot module that adds health and info endpoints for monitoring. We expose only health and info.

Q24. What does `kubectl apply` do?
Makes the cluster match the YAML: creates objects that don't exist, updates ones that changed, leaves the rest alone.

Q25. What would you improve?
A database instead of memory, readiness probes and resource limits, HTTPS with an Ingress, a webhook trigger, Workload Identity instead of a JSON key, and image vulnerability scanning.

Q26. What does `-B -ntp` mean in the Maven command?
Batch mode (no prompts) and no transfer progress (hide download lines) so the CI log is clean.

Q27. Why is the image tag the build number?
Each build gets a unique, traceable tag (1, 2, 3...). A new tag is also what tells Kubernetes there is a new version to roll out.

Q28. What if a pod crashes?
The Deployment sees only 1 of 2 running and starts a replacement automatically. The Service stops sending traffic to the dead pod.

15. GitHub Actions (the actions branch)

We have **two pipelines that do the same thing**:

- **Jenkins** builds and deploys pushes to the `main` branch (needs your laptop on).
- **GitHub Actions** builds and deploys pushes to the `actions` branch (runs in GitHub's cloud, laptop not needed).

They never clash: the Jenkins job only watches `*/main`, and the workflow only triggers on `actions`. Both run the **same** `ansible/deploy.sh` and `ansible/deploy.yml`, so they deploy in exactly the same way. The workflow file is `.github/workflows/ci-cd.yml`, and it only exists on the `actions` branch.

ci-cd.yml line by line

#	Line	Meaning
1	`name: CI/CD`	The workflow's name in the GitHub Actions tab.

4-6	`on:`
	`  push:`
	`  branches: [actions]`
		Run only when someone pushes to the `actions` branch.

8-10	`env:` `  IMAGE: asia-south1-docker.pkg.dev/cia2-513606/library-manager/library-manager:gha-${{ github.run_number }}`	The image name. `github.run_number` counts runs (1, 2, 3...). The `gha-` prefix keeps these tags separate from Jenkins' tags.
13-14	`build-and-deploy:` `  runs-on: ubuntu-latest`	One job, run on a fresh **Ubuntu VM in GitHub's cloud** (the "runner"). Docker is already installed on it.
16-17	`- name: Checkout` `  uses: actions/checkout@v4`	Copy the repo onto the runner (same as Jenkins' Checkout SCM).

19-24	`uses: actions/setup-java@v4`
	`  distribution: temurin`
	`  java-version: '21'`
	`  cache: maven`
		Install Java 21 on the runner, and cache Maven downloads between runs so later builds are faster.

26-27	`- name: Build and Test` `  run: mvn -B -ntp clean package`	Same Maven command as Jenkins: compile, 21 tests, jar.

29-32	`env:`
	`  GCP_CREDENTIALS: ${{ secrets.GCP_CREDENTIALS }}`
	`run: printf '%s' "$GCP_CREDENTIALS" > "$RUNNER_TEMP/key.json"`
		Take the JSON key from the GitHub secret `GCP_CREDENTIALS` and write it to a temporary file. GitHub hides secret values in logs. `RUNNER_TEMP` is a temp folder deleted after the job.

34-38	`docker build -t "$IMAGE" .`
	`docker login -u _json_key --password-stdin https://asia-south1-docker.pkg.dev < "$RUNNER_TEMP/key.json"`
	`docker push "$IMAGE"`
		Build, log in, push: same as Jenkins. `< file` feeds the key file in as the password (Linux's version of Jenkins' `type ... |`).

40-41	`- name: Deploy with Ansible` `  run: docker run --rm -v "$PWD:/work" -v "$RUNNER_TEMP/key.json:/key.json" -w /work google/cloud-sdk:slim bash ansible/deploy.sh "$IMAGE"`	Same container and script as Jenkins. `$PWD` is the repo folder on the runner (Jenkins uses `%WORKSPACE%`).

Jenkins vs GitHub Actions

	Jenkins (main)	GitHub Actions (actions)
Runs on	Your Windows laptop	GitHub's temporary Ubuntu VM
Trigger	Polls GitHub every 2 min (or webhook)	Instant on push (GitHub triggers it itself)
Commands	`bat` (Windows), `%VAR%`	`run` (Linux bash), `$VAR`
Key comes from	Jenkins credential `gcp-key`	GitHub secret `GCP_CREDENTIALS`
Image tag	`1, 2, 3...`	`gha-1, gha-2...`
Deploy	Identical: same container, deploy.sh and playbook	

The first `actions` run was cancelled on purpose. It had already passed the tests and pushed image `gha-1`, but it stopped while installing tools, before it touched GKE.

16. Deeper questions explained

16.1 Where does Ansible actually run?

Inside a temporary Linux container, on whichever machine runs the pipeline. It never runs on Google Cloud.

```
YOUR LAPTOP (Jenkins)                      or   GITHUB'S CLOUD (Actions)
 Jenkins (Windows program)                        Ubuntu runner VM
  -> Docker Desktop                                -> Docker
      -> google/cloud-sdk container                    -> google/cloud-sdk container
          -> Ansible  --kubectl over internet-->  GKE cluster (Mumbai) -> 2 pods
```

- **Jenkins does not run Ansible itself.** Jenkins only starts the container with `docker run`, waits, and shows the container's output in its log. That is why `PLAY RECAP` appears in the Jenkins console.
- **Why a container:** Ansible can't run on Windows, and the `google/cloud-sdk` image already has `gcloud`.
- **"localhost" in the inventory = the container.** Ansible does not SSH anywhere. Its tasks run `kubectl`, and kubectl talks to GKE's API over the internet.
- The container is deleted after each run (`--rm`), so nothing stays installed.

> **Viva:** "Jenkins triggers Ansible, but Ansible runs in a Linux container that Jenkins starts on the same machine, and its kubectl commands deploy to GKE remotely."

16.2 What is deploy.sh for?

It **prepares the empty container** and then starts Ansible. Four jobs: (1) copy the repo to `/tmp/app` so Ansible may edit files, (2) install Ansible, kubectl and the GKE auth plugin, (3) **get permission and connect**: log in with the key and point kubectl at the cluster, (4) run the playbook with the image name. Ansible does not go "into" Kubernetes. It sends commands from outside, like giving instructions over the phone; the key is the visitor pass.

16.3 What does get-credentials do?

`gcloud container clusters get-credentials cia2-gke --region asia-south1 --project cia2-513606` sets up kubectl to talk to our cluster:

1. Asks Google Cloud for the cluster's **API address** and **CA certificate**.
2. Writes them into kubectl's config file `~/.kube/config` as the entry `gke_cia2-513606_asia-south1_cia2-gke`.
3. Makes that the current entry, so plain `kubectl ...` goes to this cluster.
4. Sets kubectl to log in through `gke-gcloud-auth-plugin`, which gets a short-lived token for the logged-in service account each time.

No password is downloaded. The permission comes from the service account's **Kubernetes Engine Developer** role.

16.4 How does the image name get from Jenkins to Ansible ($1)?

```
BUILD_NUMBER = 3                                        (set by Jenkins)
 -> IMAGE = ...library-manager:3                         (Jenkinsfile environment)
 -> bash ansible/deploy.sh %IMAGE%                       (Jenkinsfile deploy stage)
 -> $1 inside deploy.sh                                  (first argument of the script)
 -> ansible-playbook ... -e image="$1"                   (extra variable for Ansible)
 -> {{ image }} in deploy.yml                            (replace task writes it into deployment.yaml)
 -> kubectl apply                                        (Kubernetes runs pods with image :3)
```

Where to see it: Jenkins → cia2-pipeline → build #3 → **Console Output**, search for `deploy.sh`. The command is shown with the value filled in, ending in `...library-manager:3`. Also visible in **Stages** → **Deploy with Ansible**. On GitHub Actions: the run → Deploy with Ansible step.

16.5 Where exactly is the image pushed?

In the **Docker Build and Push** stage. `docker build -t %IMAGE% .` names the image with the full registry address (the name is what tells Docker *where* it goes). `docker login` with the key gives permission. `docker push %IMAGE%` uploads it. In the log you see `Login Succeeded`, then `Pushed` lines, then `digest: sha256:...`, which confirms the image is in the registry.

16.6 How does the IP on port 80 reach the app on port 8080?

1. The browser sends `http://35.200.213.28` to port 80 (the default for http).
2. `type: LoadBalancer` made GKE create a **Google Cloud load balancer** that owns that IP and accepts port 80 (`port: 80`).
3. Kubernetes keeps a live list of pods with the label `app: library-manager` (the Service's **Endpoints**) and picks one.
4. Network rules on the node send the request to that **pod's IP on port 8080** (`targetPort: 8080`).
5. Spring Boot, listening on 8080, answers. The `host` in the reply shows which pod answered.

```
Browser :80 -> Load balancer 35.200.213.28:80 -> Service (port 80) -> Pod IP:8080 -> Spring Boot
```

The app never needs to run on port 80, and users never type `:8080`. See the live pod list with `kubectl describe service library-manager` (Endpoints line).

16.7 How is an old deployment replaced?

1. Ansible changes `image: ...:2` to `image: ...:3` in deployment.yaml.
2. `kubectl apply` sees the Deployment **already exists**, so it **updates** it. Only the image is different.
3. A new image = a new pod template, so Kubernetes creates a **new ReplicaSet** (the object that manages pods for one version). The old ReplicaSet stays.
4. **Rolling update**, with at most 1 extra pod and 0 unavailable:

```
Start:   old-A ok   old-B ok                (2 pods, image :2)
Step 1:  old-A ok   old-B ok   new-A starting
Step 2:  new-A ready
Step 3:  old-A removed       -> old-B, new-A
Step 4:  new-B starting      -> old-B, new-A, new-B
Step 5:  old-B removed       -> new-A, new-B (2 pods, image :3) - done
```

- At least 2 pods can always serve requests, so there is **no downtime**.
- The Service routes by label, so traffic moves to new pods automatically. The **IP never changes**.
- `kubectl rollout status` waits for the end, then Ansible checks health.
- **Rollback:** the old ReplicaSet is kept at 0 pods. `kubectl rollout undo deployment/library-manager` brings `:2` back the same rolling way.
- **Same image again?** `apply` sees no change and nothing restarts. That is why every build uses a new tag.
- **Caveat:** with no readiness probe, a pod counts as ready as soon as its container starts, a few seconds before Spring Boot is up, so a request could briefly fail. A `readinessProbe` on `/actuator/health/readiness` fixes that.

16.8 Why Ansible, if Jenkins could run the same commands?

True: Jenkins could run the kubectl commands directly. For a project this small Ansible is not strictly needed. Reasons we still use it:

- **The assignment asks for it** ("Ansible ... for deployment automation").
- **Not locked to one CI tool:** the same playbook runs from Jenkins *and* GitHub Actions. Written once, reused twice. Jenkins and Actions decide *when* to deploy; Ansible decides *how*.
- **Runs without any CI tool:** `ansible-playbook -i ansible/inventory.ini ansible/deploy.yml -e image=...` does the same deploy by hand.
- **Built-in features:** `until/retries/delay` instead of hand-written loops, modules like `replace` and `uri`, and a clear PLAY RECAP.
- **Readable:** each task has a plain-English name.
- **Scales:** Ansible's main use is configuring many servers over SSH (install Java, Docker, Jenkins...). The tool is already in place if the project grows.

16.9 Does my laptop need to be on?

For the Jenkins pipeline, yes: Jenkins (auto-starts with Windows), **Docker Desktop** (must be open) and internet are all needed. A push made while the laptop is off is not lost: when it is back on, Jenkins polls within 2 minutes and builds it.

For the running app, no: it runs on Google's machines and stays up while the cluster, pods and Service exist. **For GitHub Actions, no:** it runs in GitHub's cloud. In production, Jenkins would run on an always-on server.

16.10 Why does our Jenkinsfile have a PATH line when other Jenkinsfiles don't?

Many Jenkinsfiles (e.g. with `sh 'mvn ...'` and Linux paths like `/srv/git/...`) run on a **Linux** Jenkins where Maven and Docker are installed system-wide in `/usr/bin`, which every user's PATH includes. Ours runs on **Windows as a service under LocalSystem**, and Maven and Docker are only on *your user's* PATH, which LocalSystem can't see. So the Jenkinsfile adds them. Cleaner options: a Jenkins-managed Maven (**Manage Jenkins** → **Tools** + `tools { maven 'M3' }`), a global PATH in **Manage Jenkins** → **System** → **Global properties**, or adding the folders to the Windows system PATH.

17. Jenkins webhook (instant builds)

Polling checks GitHub every 2 minutes. A **webhook** makes GitHub call Jenkins the moment you push. Problem: Jenkins is on a laptop with no public address, so we use **ngrok** to create a temporary public URL. The Jenkins **GitHub plugin** (already installed) provides the endpoint `/github-webhook/`.

1. **Expose Jenkins:** run `ngrok http 8080` and copy the `https://...ngrok-free.app` URL. (First time: `ngrok config add-authtoken <token>` from a free ngrok account.)
2. **Add the webhook on GitHub:** repo → Settings → Webhooks → Add webhook. Payload URL `<ngrok URL>/github-webhook/` (trailing slash needed), content type `application/json`, "Just the push event". A green tick means the ping worked.
3. **Change the trigger** in the Jenkinsfile, then commit and push to main:

   ```
   triggers {
       githubPush()
   }
   ```

4. Click **Build Now** once so Jenkins registers the new trigger.
5. **Test:** push to main. The build starts within seconds and says "*Started by GitHub push by pranavsurya77*".

- **Only main builds:** GitHub calls the webhook for every branch, but the job's branch is `*/main`, so Jenkins ignores pushes to `actions`.
- The free ngrok URL changes each restart (update it in GitHub), unless you use ngrok's one free static domain.
- The tunnel makes Jenkins public. Your login still protects it; stop ngrok when not in use. You can keep `pollSCM` as a fallback.

18. Command cheat sheet

Command	Use
`kubectl get deployment library-manager`	READY count (e.g. 2/2)
`kubectl describe deployment library-manager`	Image, replicas, rolling-update settings, events
`kubectl rollout history deployment/library-manager`	Past versions
`kubectl rollout undo deployment/library-manager`	Roll back to the previous version
`kubectl rollout restart deployment/library-manager`	Restart pods one at a time
`kubectl set image deployment/library-manager library-manager=<image>`	Switch image by hand (second name = container name)
`kubectl describe service library-manager`	Shows Endpoints: the pod IPs getting traffic
`kubectl get pods -l app=library-manager -o wide`	Our pods with their IPs and nodes
`kubectl logs -l app=library-manager`	App logs from all our pods
`kubectl get deployment,service,pods -l app=library-manager`	Everything at once
`git push -u origin actions`	Run the GitHub Actions pipeline
`mvn clean package`	Build and run tests locally
`java -jar target/library-manager.jar --server.port=8081`	Run the app locally (8081 because Jenkins uses 8080)
`docker build -t library-manager:local .`	Build the image locally

`git log --oneline`	Show the 3 commits
`kubectl get deployment,pods,service`	See the deployment, the 2 pods and the service IP
`kubectl rollout status deployment/library-manager`	Wait for an update to finish
`kubectl scale deployment library-manager --replicas=0`	Stop the pods (saves money)
`kubectl delete service library-manager`	Remove the load balancer
`curl http://<IP>/actuator/health`	Health check, expect `{"status":"UP"}`
`ansible-playbook -i ansible/inventory.ini ansible/deploy.yml -e image=<image>`	Run the deploy playbook by hand